

SORTING

PROBLEM 9

QUESTION:

DS	Session	Sorting	Question Information	Level 1	Challenge 19
Problem Description: Banana leaf platter is a traditional method of serving rice dishes in South Indian cuisine. Due to the migration of South Indians, banana leaf rice can also be found in areas with significant ethnic South Indian diaspora such as Malaysia and Singapore. Irfan is a banana leaf sales person. he has N stacks of banana leaves. Each stack contains K leaves. Each leaf has a positive beauty value, describing how attractive it looks. Irfan would like to take exactly P leaves to use for lunch today. If he would like to take a leaf in a stack, he must also take all of the leaves above it in that stack as well. Help Irfan pick the P leaves that would maximize the total sum of attractive values. Constraints: $1 \leq T \leq 100.$ $1 \leq K \leq 30.$ $1 \leq P \leq N * K.$ $1 \leq N \leq 50.$ Input Format: The first line of the input gives the number of test cases, T. T test cases follow. Each test case begins with a line containing the three integers N, K and					

P. Then, N lines follow. The i-th line contains K integers, describing the attractive values of each stack of leaves from top to bottom. Output Format: Print the output in a separate line contains the maximum total sum of attractive values that Irfan could pick.

Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) <pre>2 2 4 5 10 10 100 30 80 50 10 50 3 2 3 80 80 20 20 15 50</pre> EXPECTED OUTPUT <pre>250 180</pre>	INPUT (STDIN) <pre>2 2 4 5 20 20 70 30 60 30 10 50 3 2 3 60 70 15 40 20 10</pre> EXPECTED OUTPUT <pre>200 150</pre>

Mandatory Test Cases

Test Case 1	Test Case 2
KEYWORD <pre>int max(int a,int b)</pre>	KEYWORD <pre>for(int i = 0;i < n;i++)</pre>

▼ Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
2	380	47

SOURCE CODE:

```
#include <stdio.h>

int pick_higher(int u, int v) {
    return (u > v) ? u : v;
}

int main() {
    int test_instances;
    if (scanf("%d", &test_instances) != 1) return 0;

    while (test_instances--) {
        int n_stacks, stack_sz, required_leaves;
        scanf("%d %d %d", &n_stacks, &stack_sz, &required_leaves);

        int memo[1005] = {0};
        for (int s = 0; s < n_stacks; s++) {
            int prefix_sums[35] = {0};
            for (int k = 1; k <= stack_sz; k++) {
                int beauty; scanf("%d", &beauty);
                prefix_sums[k] = prefix_sums[k - 1] + beauty;
            }
            for (int cap = required_leaves; cap >= 0; cap--) {
                for (int take = 1; take <= stack_sz && take <= cap; take++) {
                    memo[cap] = pick_higher(memo[cap], memo[cap - take] + prefix_sums[take]);
                }
            }
            printf("%d\n", memo[required_leaves]);
        }
        return 0;
    }
}
```

OUTPUT:

```
2
2 4 5
10 10 100 30
80 50 10 50
3 2 3
80 80
20 20
20 10
250
180
```

```
2
2 4 5
20 20 70 30
60 30 10 50
3 2 3
60 70
15 40
20 10
200
150
```